

DBMS

MINI PROJECT - REVIEW 3

NAME: AMOGH SUNIL

SRN: PES2UG23CS057

NAME: AARUSH R LOBO

SRN: PES2UG23CS011

DATABASE MANAGEMENT SYSTEM - REVIEW 3

TRIGGERS, PROCEDURES AND FUNCTIONS

Course Code: UE23CS351A

Mini Project Title: Exotic Animal Database Management System

TABLE OF CONTENTS

1. [Functions](#)
 - 1.1 Get Animal Age in Years
 - 1.2 Calculate Total Medical Cost
 - 1.3 Health Risk Score Calculator
 2. [Stored Procedures](#)
 - 2.1 Add New Animal
 - 2.2 Assign Primary Keeper
 - 2.3 Record Medical Visit
 3. [Triggers](#)
 - 3.1 Weight Validation (Insert)
 - 3.2 Weight Validation (Update)
 - 3.3 Facility Capacity Enforcement
 - 3.4 Primary Keeper Uniqueness (Insert)
 - 3.5 Primary Keeper Uniqueness (Update)
-

1. FUNCTIONS

1.1 Get Animal Age in Years

Purpose: Calculate the current age of an animal in years based on its birth date.

Business Logic: This function helps track animal maturity for breeding programs, dietary requirements, and age-appropriate care protocols.

Function Definition:

```
DELIMITER //
```

```
CREATE FUNCTION getAnimalAgeYears(p_animal_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE bdate DATE;
    DECLARE yrs INT;

    SELECT birth_date INTO bdate
    FROM Animals
    WHERE animal_id = p_animal_id;

    IF bdate IS NULL THEN
        SET yrs = 0;
    ELSE
        SET yrs = TIMESTAMPDIFF(YEAR, bdate, CURDATE());
    END IF;

    RETURN yrs;
END//
```

```
DELIMITER ;
```

Test Query:

```
SELECT
    animal_name,
    birth_date,
    getAnimalAgeYears(animal_id) AS age_years
FROM Animals
WHERE birth_date IS NOT NULL;
```

Expected Output:

animal_name	birth_date	age_years
Shera	2018-05-12	7
Rani	2017-03-08	8
Gajraj	2010-08-20	15
Hima	2016-12-03	9

1.2 Calculate Total Medical Cost

Purpose: Calculate the cumulative medical expenses for a specific animal across all medical records.

Business Logic: Helps facilities track healthcare budgets, identify high-cost animals, and plan for future medical expenses.

Function Definition:

```
DELIMITER //

CREATE FUNCTION totalMedicalCost(p_animal_id INT)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    DECLARE total DECIMAL(12,2);

    SELECT IFNULL(SUM(cost), 0) INTO total
    FROM MedicalRecords
    WHERE animal_id = p_animal_id;

    RETURN total;
END//

DELIMITER ;
```

Test Query:

```
SELECT
    a.animal_name,
    s.species_name,
    totalMedicalCost(a.animal_id) AS total_medical_cost
FROM Animals a
JOIN Species s ON a.species_id = s.species_id
ORDER BY total_medical_cost DESC;
```

Expected Output:

animal_name	species_name	total_medical_cost
Gajraj	African Elephant	3500.00
Rex	Komodo Dragon	2800.00
Shera	Bengal Tiger	2500.00
Hima	Snow Leopard	2200.00

1.3 Health Risk Score Calculator

Purpose: Calculate a comprehensive health risk score based on conservation status and recent emergency medical events.

Business Logic: Combines conservation vulnerability (1-5 points) with recent emergency/surgery count (up to 3 points) to prioritize care and monitoring resources.

Function Definition:

```
DELIMITER //
```

```
CREATE FUNCTION healthRiskScore(p_animal_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE status VARCHAR(50);
    DECLARE base INT DEFAULT 0;
    DECLARE recent_emergencies INT DEFAULT 0;

    SELECT s.conservations_status INTO status
    FROM Animals a
    JOIN Species s ON a.species_id = s.species_id
    WHERE a.animal_id = p_animal_id;

    IF status IN ('Critically Endangered') THEN SET base = 5;
    ELSEIF status IN ('Endangered') THEN SET base = 4;
    ELSEIF status IN ('Vulnerable') THEN SET base = 3;
    ELSEIF status IN ('Near Threatened') THEN SET base = 2;
    ELSE SET base = 1;
    END IF;

    SELECT COUNT(*) INTO recent_emergencies
    FROM MedicalRecords
    WHERE animal_id = p_animal_id
       AND record_type IN ('Emergency', 'Surgery')
       AND visit_date >= DATE_SUB(CURDATE(), INTERVAL 180 DAY);

    RETURN base + LEAST(recent_emergencies, 3);
END//
```

```
DELIMITER ;
```

Test Query:

```
SELECT
    a.animal_name,
```

```

    s.species_name,
    s.conservation_status,
    healthRiskScore(a.animal_id) AS risk_score
FROM Animals a
JOIN Species s ON a.species_id = s.species_id
ORDER BY risk_score DESC;
```

Expected Output:

animal_name	species_name	conservation_status	risk_score
Rex	Komodo Dragon	Vulnerable	4
Shera	Bengal Tiger	Endangered	4
Rani	Bengal Tiger	Endangered	4
Ruby	Red Panda	Endangered	4

2. STORED PROCEDURES

2.1 Add New Animal

Purpose: Add a new animal to the database with comprehensive validation checks.

Business Logic: Ensures data integrity by validating species/facility existence, checking facility capacity, and preventing overcrowding.

Procedure Definition:

```

DELIMITER //

CREATE PROCEDURE addAnimal(
    IN p_animalname VARCHAR(100),
    IN p_speciesid INT,
    IN p_facilityid INT,
    IN p_gender ENUM('Male', 'Female', 'Unknown'),
    IN p_birthdate DATE,
    IN p_acquisitiondate DATE,
    IN p_acquisitionmethod ENUM('Born in
facility', 'Purchased', 'Rescued', 'Transferred', 'Donated'),
    IN p_healthstatus ENUM('Excellent', 'Good', 'Fair', 'Poor', 'Critical'),
    IN p_weightkg DECIMAL(8,2),
    IN p_microchipid VARCHAR(20)
)
BEGIN
    DECLARE cap INT;
    DECLARE current_count INT;
```

```

-- Validate species exists
IF (SELECT COUNT(*) FROM Species WHERE species_id = p_speciesid) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid speciesid';
END IF;

-- Validate facility exists
IF (SELECT COUNT(*) FROM Facilities WHERE facility_id = p_facilityid) = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid facilityid';
END IF;

-- Check facility capacity
SELECT capacity INTO cap FROM Facilities WHERE facility_id = p_facilityid;
SELECT COUNT(*) INTO current_count
FROM Animals
WHERE facility_id = p_facilityid AND is_active = TRUE;

IF current_count >= cap THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Facility capacity exceeded';
END IF;

-- Insert new animal
INSERT INTO Animals(
    animal_name, species_id, facility_id, gender, birth_date, acquisition_date,
    acquisition_method, health_status, weight_kg, microchip_id, is_active
) VALUES (
    p_animalname, p_speciesid, p_facilityid, p_gender, p_birthdate,
p_acquisitiondate,
    p_acquisitionmethod, p_healthstatus, p_weightkg, p_microchipid, TRUE
);
END//

DELIMITER ;

```

Test Call:

```

CALL addAnimal(
    'Khan',
    1,
    1,
    'Male',
    '2022-01-01',
    CURDATE(),
    'Transferred',
    'Good',
    210.00,
    'MC999'
);

-- Verify insertion
SELECT * FROM Animals WHERE microchip_id='MC999';

```

Expected Output:

```

+-----+-----+-----+-----+-----+-----+
| animal_id | animal_name | species_id | facility_id | gender | birth_date |
+-----+-----+-----+-----+-----+-----+
|      9      | Khan      |      1      |      1      | Male  | 2022-01-01 |
+-----+-----+-----+-----+-----+-----+

```

Message: 1 row affected

2.2 Assign Primary Keeper

Purpose: Assign or reassign a primary keeper to an animal, automatically marking previous assignments as inactive.

Business Logic: Maintains keeper accountability by ensuring each animal has exactly one active primary caregiver while preserving historical assignment records.

Procedure Definition:

```

DELIMITER //

CREATE PROCEDURE assignPrimaryKeeper(
    IN p_animal_id INT,
    IN p_keeper_id INT,
    IN p_notes TEXT
)
BEGIN
    -- Validate animal exists and is active
    IF (SELECT COUNT(*) FROM Animals
        WHERE animal_id = p_animal_id AND is_active=TRUE) = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid or inactive animal';
    END IF;

    -- Validate keeper exists
    IF (SELECT COUNT(*) FROM Keepers WHERE keeper_id = p_keeper_id) = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid keeper';
    END IF;

    -- Deactivate current primary keeper
    UPDATE AnimalCare
    SET is_current = FALSE
    WHERE animal_id = p_animal_id AND is_current = TRUE;

    -- Insert new primary keeper assignment
    INSERT INTO AnimalCare(animal_id, keeper_id, assignment_date, care_type, notes,
is_current)
    VALUES (p_animal_id, p_keeper_id, CURDATE(), 'Primary', p_notes, TRUE);
END//

```

```
DELIMITER ;
```

Test Call:

```
CALL assignPrimaryKeeper(1, 1, 'Reassigned for medical monitoring');

-- Verify assignment
SELECT
    ac.care_id,
    a.animal_name,
    CONCAT(k.first_name, ' ', k.last_name) AS keeper_name,
    ac.assignment_date,
    ac.care_type,
    ac.is_current,
    ac.notes
FROM AnimalCare ac
JOIN Animals a ON ac.animal_id = a.animal_id
JOIN Keepers k ON ac.keeper_id = k.keeper_id
WHERE ac.animal_id = 1
ORDER BY ac.assignment_date DESC;
```

Expected Output:

care_id	animal_name	keeper_name	assignment_date	is_cur	notes
9	Shera	Rajesh Kumar	2025-10-24	TRUE	Reassigned
1	Shera	Rajesh Kumar	2019-06-15	FALSE	Experienced

2.3 Record Medical Visit

Purpose: Record medical visits with automatic health status updates for emergencies and surgeries.

Business Logic: Streamlines medical record-keeping while implementing business rules that automatically downgrade animal health status during serious medical events.

Procedure Definition:

```
DELIMITER //

CREATE PROCEDURE recordMedicalVisit(
    IN p_animal_id INT,
    IN p_recordtype ENUM('Routine
Checkup', 'Emergency', 'Surgery', 'Vaccination', 'Injury'),
```



```

    IN p_cost DECIMAL(10,2),
    IN p_diagnosis TEXT,
    IN p_treatment TEXT,
    IN p_medications VARCHAR(500),
    IN p_followupdate DATE,
    IN p_veterinarianname VARCHAR(100)
)
BEGIN
    -- Validate animal exists
    IF (SELECT COUNT(*) FROM Animals WHERE animal_id=p_animal_id) = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid animal';
    END IF;

    -- Validate cost is non-negative
    IF p_cost < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Negative cost not allowed';
    END IF;

    -- Insert medical record
    INSERT INTO MedicalRecords(
        animal_id, visit_date, veterinarian_name, diagnosis, treatment, medications,
        follow_up_date, record_type, cost
    ) VALUES (
        p_animal_id, CURDATE(), p_veterinarianname, p_diagnosis, p_treatment,
        p_medications,
        p_followupdate, p_recordtype, p_cost
    );

    -- Update health status for emergency/surgery
    IF p_recordtype IN ('Emergency','Surgery') THEN
        UPDATE Animals
        SET health_status = 'Poor'
        WHERE animal_id = p_animal_id
        AND health_status IN ('Good','Fair','Excellent');
    END IF;
END//

DELIMITER ;

```

Test Call:

```

CALL recordMedicalVisit(
    1,
    'Emergency',
    3200.00,
    'Dehydration',
    'IV Fluids administered',
    'ORS, Electrolytes',
    DATE_ADD(CURDATE(), INTERVAL 7 DAY),
    'Dr. N Rao'
);

```

```
-- Verify medical record
SELECT * FROM MedicalRecords
WHERE animal_id=1
ORDER BY visit_date DESC
LIMIT 1;

-- Check health status update
SELECT animal_name, health_status
FROM Animals
WHERE animal_id=1;
```

Expected Output:

Medical Record:

record_id	animal_id	visit_date	record_type	cost
8	1	2025-10-24	Emergency	3200.00

Health Status:

animal_name	health_status
Shera	Poor

3. TRIGGERS

3.1 Weight Validation (Insert)

Purpose: Enforce weight constraints before inserting new animal records.

Business Logic: Prevents invalid weight entries (must be between 0.001 kg and 10,000 kg) to maintain data quality.

Trigger Definition:

```
DELIMITER //
```

```
CREATE TRIGGER trg_animals_weight_check
BEFORE INSERT ON Animals
FOR EACH ROW
BEGIN
    IF NEW.weight_kg IS NOT NULL
        AND (NEW.weight_kg < 0.001 OR NEW.weight_kg > 10000) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Weight out of allowed bounds';
END
```

```
END IF;  
END//  
  
DELIMITER ;
```

Test Query:

```
-- This should FAIL  
INSERT INTO Animals(  
    animal_name, species_id, facility_id, gender,  
    acquisition_date, acquisition_method, health_status,  
    weight_kg, microchip_id, is_active  
) VALUES (  
    'TestAnimal', 1, 1, 'Male',  
    CURDATE(), 'Transferred', 'Good',  
    -5, 'MCTEST001', TRUE  
);
```

Expected Output:

```
ERROR 1644 (45000): Weight out of allowed bounds
```

3.2 Weight Validation (Update)

Purpose: Enforce weight constraints before updating existing animal records.

Business Logic: Maintains data integrity by preventing invalid weight updates during animal record modifications.

Trigger Definition:

```
DELIMITER //  
  
CREATE TRIGGER trg_animals_weight_check_upd  
BEFORE UPDATE ON Animals  
FOR EACH ROW  
BEGIN  
    IF NEW.weight_kg IS NOT NULL  
        AND (NEW.weight_kg < 0.001 OR NEW.weight_kg > 10000) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Weight out of allowed bounds';  
    END IF;  
END//  
  
DELIMITER ;
```

Test Query:

```
-- This should FAIL
UPDATE Animals
SET weight_kg = -5
WHERE animal_id = 1;
```

Expected Output:

```
ERROR 1644 (45000): Weight out of allowed bounds
```

3.3 Facility Capacity Enforcement

Purpose: Prevent exceeding facility capacity when adding new animals.

Business Logic: Automatically checks active animal count against facility capacity before allowing new animal insertions.

Trigger Definition:

```
DELIMITER //
```

```
CREATE TRIGGER trg_facility_capacity
BEFORE INSERT ON Animals
FOR EACH ROW
BEGIN
    DECLARE cap INT;
    DECLARE cnt INT;

    SELECT capacity INTO cap
    FROM Facilities
    WHERE facility_id = NEW.facility_id;

    SELECT COUNT(*) INTO cnt
    FROM Animals
    WHERE facility_id = NEW.facility_id AND is_active = TRUE;

    IF cnt >= cap THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Facility capacity exceeded (trigger)';
    END IF;
END//
```

```
DELIMITER ;
```

Test Query:

```
-- Temporarily reduce capacity to test
UPDATE Facilities SET capacity = 1 WHERE facility_id = 1;

-- This should FAIL (capacity exceeded)
INSERT INTO Animals(
    animal_name, species_id, facility_id, gender,
    acquisition_date, acquisition_method, health_status,
    weight_kg, microchip_id, is_active
) VALUES (
    'TestCapacity', 1, 1, 'Male',
    CURDATE(), 'Transferred', 'Good',
    100.00, 'MCCAP001', TRUE
);

-- Restore capacity
UPDATE Facilities SET capacity = 500 WHERE facility_id = 1;
```

Expected Output:

```
ERROR 1644 (45000): Facility capacity exceeded (trigger)
```

3.4 Primary Keeper Uniqueness (Insert)

Purpose: Ensure each animal has only one active primary keeper at any given time.

Business Logic: Prevents data inconsistencies by blocking attempts to assign multiple primary keepers to the same animal simultaneously.

Trigger Definition:

```
DELIMITER //
```

```
CREATE TRIGGER trg_animalcare_primary_uniqueness
BEFORE INSERT ON AnimalCare
FOR EACH ROW
BEGIN
    IF NEW.is_current = TRUE AND NEW.care_type = 'Primary' THEN
        IF EXISTS (
            SELECT 1 FROM AnimalCare
            WHERE animal_id = NEW.animal_id
            AND is_current = TRUE
            AND care_type = 'Primary'
        ) THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Primary current keeper already exists for this animal';
        END IF;
    END IF;
```

```
END//  
  
DELIMITER ;
```

Test Query:

```
-- This should FAIL (primary keeper already exists)  
INSERT INTO AnimalCare(  
    animal_id, keeper_id, assignment_date,  
    care_type, notes, is_current  
) VALUES (  
    1, 2, CURDATE(),  
    'Primary', 'Conflict test', TRUE  
);
```

Expected Output:

```
ERROR 1644 (45000): Primary current keeper already exists for this animal
```

3.5 Primary Keeper Uniqueness (Update)

Purpose: Maintain primary keeper uniqueness constraint during record updates.

Business Logic: Prevents accidentally creating duplicate primary keeper assignments when modifying existing care records.

Trigger Definition:

```
DELIMITER //
```



```
CREATE TRIGGER trg_animalcare_primary_uniqueness_upd  
BEFORE UPDATE ON AnimalCare  
FOR EACH ROW  
BEGIN  
    IF NEW.is_current = TRUE AND NEW.care_type = 'Primary' THEN  
        IF EXISTS (  
            SELECT 1 FROM AnimalCare  
            WHERE animal_id = NEW.animal_id  
                AND is_current = TRUE  
                AND care_type = 'Primary'  
                AND care_id <> NEW.care_id  
        ) THEN  
            SIGNAL SQLSTATE '45000'  
            SET MESSAGE_TEXT = 'Primary current keeper already exists for this animal';  
        END IF;  
    END IF;
```

```
END//  
  
DELIMITER ;
```

Test Query:

```
-- First, set a care record to inactive  
UPDATE AnimalCare SET is_current = FALSE WHERE care_id = 1;  
  
-- Now try to reactivate it (should FAIL if another primary exists)  
UPDATE AnimalCare  
SET is_current = TRUE, care_type = 'Primary'  
WHERE care_id = 1;
```

Expected Output:

```
ERROR 1644 (45000): Primary current keeper already exists for this animal
```

SUMMARY

Functions Implemented: 3

1. **getAnimalAgeYears** - Age calculation
2. **totalMedicalCost** - Medical expense tracking
3. **healthRiskScore** - Risk assessment

Procedures Implemented: 3

1. **addAnimal** - Animal registration with validation
2. **assignPrimaryKeeper** - Keeper assignment management
3. **recordMedicalVisit** - Medical record creation with auto-updates

Triggers Implemented: 5

1. **trg_animals_weight_check** - Weight validation (Insert)
2. **trg_animals_weight_check_upd** - Weight validation (Update)
3. **trg_facility_capacity** - Capacity enforcement
4. **trg_animalcare_primary_uniqueness** - Keeper uniqueness (Insert)
5. **trg_animalcare_primary_uniqueness_upd** - Keeper uniqueness (Update)

Declaration: We hereby declare that the work presented in this report is original and has been completed by the team members mentioned above.

Date: October 24, 2025

Signatures:

- Aarush Ryan Lobo (PES2UG23CS011)
- AMOGH SUNIL (PES2UG23CS057)